

Les procédures stockées MYSQL

Les Procédures Stockées

Définition :

Une **procédure stockée** est un petit programme stocké dans la base de données et appellable à partir d'un client comme on peut le faire pour une requête. Une procédure est exécutée par le serveur de base de données.

Lorsque l'on crée une procédure, un des premiers soucis à contourner : comment écrire une instruction qui se termine par **un point-virgule** alors que ma procédure contient un point-virgule à chaque ligne.

La réponse est : changer le caractère délimiteur (**DELIMITER**) par un autre qui ne risque pas d'apparaître dans nos instructions, prenons par exemple le délimiteur **&&**.

Création d'une procédure stockée :

La syntaxe suivante est utilisée pour créer une procédure stockée dans MySQL :

```
DROP PROCEDURE IF EXISTS procedure_name;
DELIMITER &&
CREATE PROCEDURE procedure_name ([IN | OUT | INOUT] parameter datatype,...)
BEGIN
    Declaration_section
    Executable_section
END &&
DELIMITER ;
```

La syntaxe de la procédure a les paramètres suivants :

- **procedure_name** : représente le nom de la procédure stockée.
- **parameter** : représente les paramètres de la procédure stockée.
- **Declaration_section** : représente les déclarations de toutes les variables.
- **Executable_section** : représente le code pour l'exécution de la procédure.

Trois types de paramètres pour les procédures stockées :

- **IN** : C'est le mode par défaut. Il prend un paramètre en entrée, tel qu'un attribut. Lorsque nous le définissons, le programme appelant doit passer un argument à la procédure stockée. La valeur de ce paramètre est toujours protégée.
- **OUT** : Il est utilisé pour passer un paramètre en sortie. Sa valeur peut être modifiée dans la procédure stockée et la valeur modifiée (nouvelle) est renvoyée au programme appelant.
- **INOUT** : C'est une combinaison des paramètres IN et OUT. Cela signifie que le programme appelant peut transmettre l'argument et que la procédure peut modifier le paramètre INOUT, puis retransmet la nouvelle valeur au programme appelant.

Remarque : on peut aussi commencer la déclaration de la procédure par la syntaxe :

```
CREATE PROCEDURE [IF NOT EXISTS] procedure_name(...)
```

Appeler une procédure stockée

Nous pouvons utiliser l'instruction **CALL** pour appeler une procédure stockée :

```
CALL procedure_name ( parameter(s))
```

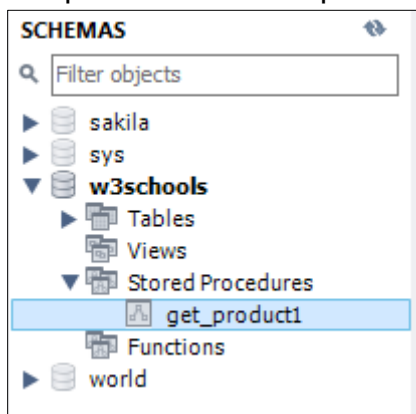
Exemple de procédure stockée sans paramètre

On veut créer une procédure qui affiche tous les enregistrements de la table **products** dont les prix sont supérieurs à 40 et compter après toutes les lignes de la table :

```
DELIMITER &&
CREATE PROCEDURE get_product1()
BEGIN
    SELECT * FROM products WHERE Price > 40;
    SELECT COUNT(*) AS Total FROM products;
END &&
DELIMITER ;

CALL get_product1();
```

Les procédures créées par l'utilisateur se trouvent dans le dossier **Stored Procedures** :



Exemple de procédure stockée avec paramètre IN

L'exemple suivant crée une procédure stockée qui affiche les **n** premières lignes de la table **products** :

```
DELIMITER $$
CREATE PROCEDURE get_products(IN n INT)
BEGIN
    SELECT * FROM products LIMIT n;
END $$
DELIMITER ;

CALL get_products(10);
```

Exemple de procédure stockée avec paramètre OUT

L'exemple suivant crée une procédure stockée qui compte le nombre de produit qui ont un prix égal à 10 et stocke le résultat dans la variable de sortie **Total** :

```
DELIMITER $$
CREATE PROCEDURE display_products(OUT Total INT)
BEGIN
    SELECT COUNT(*) INTO Total FROM products WHERE Price = 10;
END $$
DELIMITER ;

CALL display_products(@T);
SELECT @T AS Total;
```

Exemple de procédure stockée avec paramètre INOUT

L'exemple suivant crée une procédure stockée qui prend comme paramètre la variable d'entrée sortie **var1** qui prend comme valeur d'entrée le **ProductID** et qui affiche aussi dans la variable **var1** comme sortie la prix de ce produit :

```
DELIMITER $$
CREATE PROCEDURE get_price(INOUT var1 INT)
BEGIN
    SELECT Price INTO var1 FROM products WHERE ProductID = var1;
END $$
DELIMITER ;

SET @V = 1;
CALL get_price(@V);
SELECT @V AS Price;
```

Lister les procédures stockées dans MySQL

Lorsque nous avons plusieurs procédures dans le serveur MySQL, il est très important de lister toutes les procédures. Car parfois les noms de procédure sont les mêmes dans de nombreuses bases de données.

Nous pouvons lister toutes les procédures stockées sur le serveur MySQL actuel comme suit :

```
SHOW PROCEDURE STATUS;
```

	Db	Name	Type	Definer	Modified	Created	Security_type	Comment	character_set_client	collation_connection	Database Collation
▶	sakila	film_in_stock	PROCEDURE	root@localhost	2023-01-02 15:27:40	2023-01-02 15:27:40	DEFINER		utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...
	sakila	film_not_in_stock	PROCEDURE	root@localhost	2023-01-02 15:27:40	2023-01-02 15:27:40	DEFINER		utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...
	sakila	rewards_report	PROCEDURE	root@localhost	2023-01-02 15:27:40	2023-01-02 15:27:40	DEFINER	Provides a customizabl...	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...
	sys	create_synonym_db	PROCEDURE	mysql.sys@local...	2023-01-02 15:21:15	2023-01-02 15:21:15	INVOKER	Description ----- ...	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...
	sys	diagnostics	PROCEDURE	mysql.sys@local...	2023-01-02 15:21:15	2023-01-02 15:21:15	INVOKER	Description ----- ...	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...
	sys	execute_prepared_stmt	PROCEDURE	mysql.sys@local...	2023-01-02 15:21:15	2023-01-02 15:21:15	INVOKER	Description ----- ...	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...
	sys	ps_setup_disable_backgr...	PROCEDURE	mysql.sys@local...	2023-01-02 15:21:15	2023-01-02 15:21:15	INVOKER	Description ----- ...	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...
	sys	ps_setup_disable_cons...	PROCEDURE	mysql.sys@local...	2023-01-02 15:21:15	2023-01-02 15:21:15	INVOKER	Description ----- ...	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...
	sys	ps_setup_disable_instru...	PROCEDURE	mysql.sys@local...	2023-01-02 15:21:15	2023-01-02 15:21:15	INVOKER	Description ----- ...	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...
	sys	ps_setup_disable_thread	PROCEDURE	mysql.sys@local...	2023-01-02 15:21:15	2023-01-02 15:21:15	INVOKER	Description ----- ...	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...
	sys	ps_setup_enable_backgr...	PROCEDURE	mysql.sys@local...	2023-01-02 15:21:15	2023-01-02 15:21:15	INVOKER	Description ----- ...	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...

Pour afficher les procédures stockées relative à une base de données spécifique :

```
SHOW PROCEDURE STATUS WHERE Db = 'w3schools';
SHOW PROCEDURE STATUS WHERE Db LIKE '%w3%';
```

On peut Aussi lister la liste des procédures stockées en utilisant la table routine à l'aide de la syntaxe suivante :

```
SELECT
    routine_name
FROM
    information_schema.routines
WHERE
    routine_type = 'PROCEDURE'
    AND routine_schema = 'database_name';
```

Exemple :

```
SELECT routine_name FROM information_schema.routines
WHERE routine_type = 'PROCEDURE' AND routine_schema = 'w3schools';
```

Supprimer une procédure stockée :

Pour supprimer une procédure stockée, utiliser la syntaxe suivante :

```
DROP PROCEDURE [IF EXISTS] procedure_name;
```

Exemple :

```
DROP PROCEDURE get_product1;
```